

SET-8 — MASTER PRACTICAL GUIDE

Vehicle Rental Management System

Windows PC • Eclipse • Git/GitHub • Maven • Tomcat • Docker

Repository	https://github.com/nikithamoturi/VehicleRentalManagementSystem.git
Coverage	26 individual task entries, each followed immediately by an answer
Exam strategy	Command-first, GUI-supported, with older-version fallbacks and examiner-proof checks

Use this as the exam-day walkthrough: open Git Bash/Command Prompt, Eclipse, Docker Desktop, and a browser side by side. For every task, perform the command, verify the result, and only then move on.

HOW TO USE THIS MASTER PDF

This guide is written for a Windows PC and is designed for practical-exam demonstration. Commands are the most version-tolerant method; GUI routes are included where Eclipse/Docker Desktop can be used. Exact menu labels can differ slightly between old and new releases, so use the command equivalent if a GUI label is missing.

Important source limitation: several exam questions refer to a “provided pom.xml”, but the uploaded question papers contain the questions, not the actual POM file. For those questions, this guide deliberately shows how to derive the exact value from the supplied POM instead of inventing coordinates, WAR names, or context paths.

1. Windows tool setup

Tool	What to verify	Windows command / GUI
Java JDK	Use a JDK, not only a JRE.	<code>java -version</code> <code>javac -version</code>
Maven	Maven available to the terminal, or Maven Wrapper exists.	<code>mvn -version</code> <code>mvnw.cmd -version</code>
Git	Git Bash/Command Prompt can run Git.	<code>git --version</code>
Eclipse	Maven integration (m2e) and Java support.	Window → Preferences → Java
Tomcat	Choose a Tomcat version compatible with the project.	Servers view → New → Apache Tomcat
Docker	Docker daemon/Desktop is running.	<code>docker version</code> Docker Desktop GUI

If `mvn` is not recognized

Either install/configure Maven and add its **bin** directory to Windows PATH, or check whether the repository contains **mvnw.cmd**. The wrapper is often the safest option for older student projects because it uses the project's expected Maven distribution.

```
where mvn
mvn -version
mvnw.cmd -version
```

Windows GUI: Control Panel / Settings → System → Advanced system settings → Environment Variables → create MAVEN_HOME=C:\apache-maven-3.x.x → edit Path → add %MAVEN_HOME%\bin → reopen terminal.

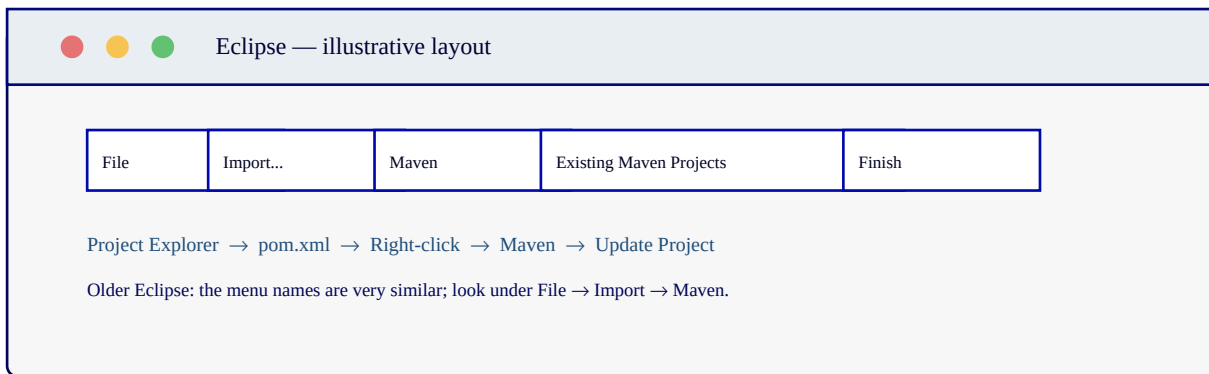
Java compatibility rule

Always run **mvn -version**, not only **java -version**, because Maven reports the Java home it actually uses. If an old project expects Java 8/11 and a newer JDK is installed, align the JDK used by Maven and Eclipse with the project's compiler settings. Do not blindly change the POM until you know the required source/target/release level.

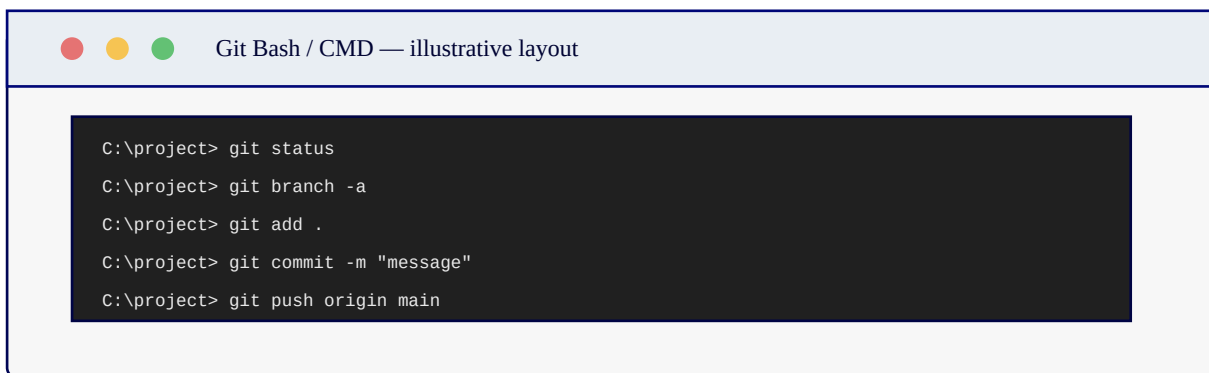
Tomcat compatibility warning

Older JSP/Servlet projects commonly use the **javax.servlet** API. Tomcat 10+ moved to the **jakarta.servlet** namespace. If the project imports `javax.servlet.*`, Tomcat 9 (or another compatible javax-based server) is usually the safer exam choice unless the code has already been migrated to Jakarta.

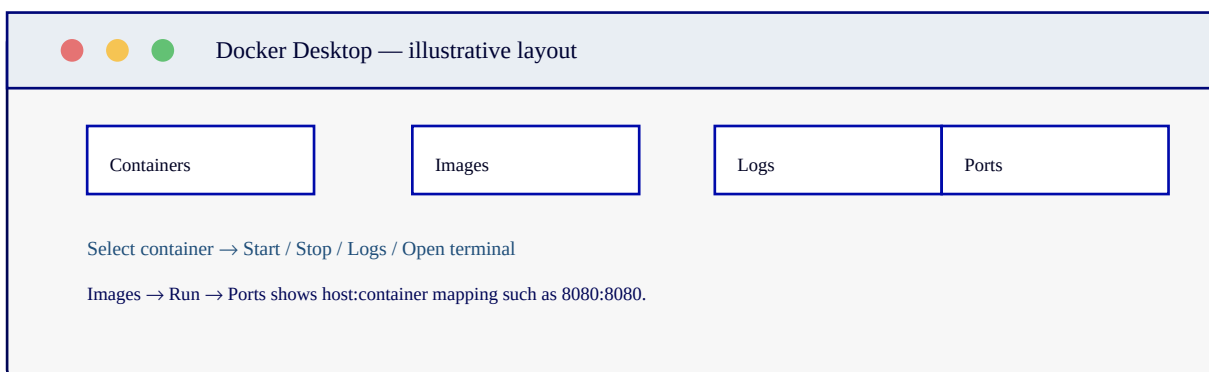
2. GUI routes you can demonstrate to the examiner



Eclipse Maven import: **File** → **Import** → **Maven** → **Existing Maven Projects** → Browse to the cloned folder → select pom.xml → Finish. Then right-click the project → Maven → Update Project. For older Eclipse releases, the wording is almost identical.



Git tasks are best demonstrated in Git Bash because every advanced command in these papers is available there. GitHub Desktop/Eclipse EGit can perform many common operations, but rebase, reflog, patch, and detailed recovery are much easier to prove from Git Bash.



Docker Desktop is useful for showing Images, Containers, Logs, and port mappings. If the lab machine has an older Docker Desktop layout, the same objects may appear under older tabs/menu names. The CLI remains the most reliable fallback.

3. Universal exam command sheet

Area	Commands
Git basics	<code>git clone URL</code> <code>cd project</code> <code>git status</code> <code>git remote -v</code> <code>git branch -a</code>
Branching	<code>git switch -c feature/name</code> # older Git: <code>git checkout -b feature/name</code>
Inspect changes	<code>git diff</code> <code>git diff --cached</code> <code>git status</code>
Commit	<code>git add file</code> <code>git commit -m "message"</code> <code>git log -1 --oneline</code>
Remote sync	<code>git fetch origin</code> <code>git merge origin/main</code> <code>git pull origin main</code>
Rebase	<code>git fetch origin</code> <code>git rebase origin/main</code> <code>git rebase --abort</code>
Undo safely	<code>git revert <hash></code> <code>git reset --soft HEAD~1</code> <code>git reflog</code>
Stash	<code>git stash push -m "work"</code> <code>git stash list</code> <code>git stash pop</code>
Compare/history	<code>git diff main feature -- file</code> <code>git log --oneline --graph --decorate --all</code>
Maven	<code>mvn -version</code> <code>mvn clean package</code> <code>mvn dependency:tree</code> <code>mvn dependency:analyze</code> <code>mvn clean package -X</code>
Tests	<code>mvn test</code> <code>mvn -Dtest=ClassName test</code> <code>mvn clean package -DskipTests</code>
Docker	<code>docker build -t app:v1 .</code> <code>docker run -d --name app -p 8080:8080 app:v1</code> <code>docker ps</code> <code>docker logs app</code>
Docker Hub	<code>docker login</code> <code>docker tag app:v1 user/app:v1</code> <code>docker push user/app:v1</code> <code>docker logout</code>

4. Universal troubleshooting decision tree

Maven failure: read the first meaningful [ERROR] → identify the phase/goal → inspect the POM coordinates/configuration → run `dependency:tree/effective-pom` → check JDK with `mvn -version` → clean → rebuild with `-X`.

Browser failure: prove Maven created the artifact → prove the server started → prove the port is listening → prove the context path → read server logs → then inspect JSP/application code.

Docker failure: prove Docker daemon → prove Dockerfile/context → prove image exists → prove container exists → inspect `docker ps -a` → inspect docker logs → inspect port mapping.

5. COMPLETE TASK-BY-TASK QUESTION → ANSWER GUIDE

The task wording below is kept close to the uploaded question paper while being shortened where a long scenario is repeated. Every numbered/sub-numbered practical requirement is represented. Where the source asks for an exact value that depends on an unseen POM, the answer explicitly tells you how to derive it.

Q0 — TASK / QUESTION

Vehicle Rental scenario: identify the required pages, data fields, deployment model, and quality goals.

ANSWER / PROCEDURE

Translate the scenario into functional and non-functional requirements, actors, core use cases, and deployment needs. For a web hospital/ticketing/recruitment system, include authentication, role-based access, CRUD workflows, reports, security, usability, and concurrent-user performance.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

Q1(a) — TASK / QUESTION

Clone VehicleRentalManagementSystem and import it into Eclipse as a Maven Web Project; verify JSP pages.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/nikithamoturi/VehicleRentalManagementSystem.git
cd VehicleRentalManagementSystem
git status
git remote -v
```

GUI route: Eclipse → File → Import → Maven → Existing Maven Projects; for an already imported project use Right-click → Maven → Update Project.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

Q1(b) — TASK / QUESTION

Identify groupId, artifactId, version, packaging, and two dependencies in pom.xml.

ANSWER / PROCEDURE

A dependency is a library your application compiles/executes against. A plugin is a Maven tool that performs build lifecycle work. Example dependency: a Servlet API or JDBC driver. Example plugin: `maven-compiler-plugin`, `maven-war-plugin`, or a Tomcat plugin.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

Q1(c) — TASK / QUESTION

Run Maven clean/build and correct dependency/compiler/configuration errors in pom.xml.

ANSWER / PROCEDURE

Open a terminal in the folder containing `pom.xml`. Run the lifecycle in small steps so the first failing phase is visible. For a deployable package, `clean package` is the normal exam command.

Commands / configuration

```
mvn -version
mvn clean
mvn compile
mvn test
mvn package
mvn clean package
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

Q1(d) — TASK / QUESTION

Verify target output, run the web application, and access the JSP pages.

ANSWER / PROCEDURE

A successful Maven build only proves the build lifecycle completed. Verify the WAR/JAR exists, start the server, confirm the server log says it started, confirm the expected port is listening, then open the correct context path. Check browser/server logs for 404/500 errors.

Commands / configuration

```
dir target
netstat -ano | findstr :8080
tasklist | findstr /I java
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

Q2(a) — TASK / QUESTION

Display the dependency tree and identify the application's dependencies.

ANSWER / PROCEDURE

The dependency tree shows direct and transitive dependencies and is the quickest way to investigate version conflicts.

Commands / configuration

```
mvn dependency:tree
mvn dependency:tree -Dverbose
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

Q2(b) — TASK / QUESTION

Explain Maven dependency resolution and the effect of version conflicts.

ANSWER / PROCEDURE

Use the dependency tree to see which paths bring different versions. Maven normally selects the nearest definition in the dependency graph; a dependencyManagement entry or explicit direct dependency can be used to pin the intended version. Exclusions remove an unwanted transitive path.

Commands / configuration

```
mvn dependency:tree -Dverbose
mvn help:effective-pom
```

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

Q2(c) — TASK / QUESTION

Locate .m2/repository and verify required JARs.

ANSWER / PROCEDURE

On Windows the default local Maven repository is `%USERPROFILE%\m2\repository`. Verify the expected group/artifact/version folder and confirm the JAR exists. Do not assume a missing JAR means the POM is wrong until you check coordinates and repositories.

Commands / configuration

```
echo %USERPROFILE%\m2\repository
mvn help:evaluate -Dexpression=settings.localRepository -q -DforceStdout
dir "%USERPROFILE%\m2\repository"
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

Q2(d) — TASK / QUESTION

Clean, compile, package, and troubleshoot a build failure.

ANSWER / PROCEDURE

Run Maven with debug logging and preserve the first meaningful error. Do not start by changing random POM values. Check the first `[ERROR]`, the goal/phase named, and the `Caused by:` chain.

Commands / configuration

```
mvn clean package -X
mvn clean package -e
mvn -version
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

Q2(e) — TASK / QUESTION

Build succeeds but JSP app fails on the server; identify whether Maven, server, or web app is at fault.

ANSWER / PROCEDURE

A successful Maven build only proves the build lifecycle completed. Verify the WAR/JAR exists, start the server, confirm the server log says it started, confirm the expected port is listening, then open the correct context path. Check browser/server logs for 404/500 errors.

Commands / configuration

```
dir target
netstat -ano | findstr :8080
tasklist | findstr /I java
```

GUI route: Eclipse → Window → Show View → Servers → add/start Tomcat; then open the URL in a browser and check the Servers/Console view for startup errors.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

Q3(a) — TASK / QUESTION

Clone Vehicle Rental and verify git status and git remote -v.

ANSWER / PROCEDURE

1) Install Git for Windows. Open Git Bash. 2) Move to the folder where you want the project. 3) Clone the repository. 4) Enter the folder and verify the remote. 5) In Eclipse use File → Import → Maven → Existing Maven Projects (older Eclipse: File → Import → Maven → Existing Maven Projects), select the folder containing pom.xml, Finish, then Project → Clean if necessary.

Commands / configuration

```
git clone https://github.com/nikithamoturi/VehicleRentalManagementSystem.git
cd VehicleRentalManagementSystem
git status
git remote -v
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q3(b) — TASK / QUESTION

Create a feature branch for the vehicle information page and verify checkout.

ANSWER / PROCEDURE

Create and switch in one operation. This is the modern command; on older Git use the checkout equivalent.

Commands / configuration

```
git switch -c feature/vehicle-update
git checkout -b feature/vehicle-update
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q3(c) — TASK / QUESTION

Modify vehicle.jsp to add Vehicle Type and Availability Status and inspect git diff.

ANSWER / PROCEDURE

Use `git status` first. It separates modified-but-unstaged files, staged files, and untracked files. For exact line-level edits use `git diff`; for staged edits use `git diff --cached`.

Commands / configuration

```
git status
git diff
git diff --cached
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q3(d) — TASK / QUESTION

Stage vehicle.jsp, commit meaningfully, and verify the commit.

ANSWER / PROCEDURE

Stage only the requested file, then create the commit. Verify the commit immediately with `git log` or `git show`.

Commands / configuration

```
git add "vehicle.jsp"
git commit -m "Add vehicle type and availability"
git log -1 --oneline
git show --stat HEAD
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q4(a) — TASK / QUESTION

Make another vehicle.jsp change, such as Rental Rate, and verify with git diff.

ANSWER / PROCEDURE

Use `git status` first. It separates modified-but-unstaged files, staged files, and untracked files. For exact line-level edits use `git diff`; for staged edits use `git diff --cached`.

Commands / configuration

```
git status
git diff
git diff --cached
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q4(b) — TASK / QUESTION

Retrieve remote changes without overwriting local commits.

ANSWER / PROCEDURE

Fetch downloads new remote objects and references but does not merge them into your current working tree. This is the safe answer when the question says 'retrieve without modifying local files'.

Commands / configuration

```
git fetch origin
git status
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q4(c) — TASK / QUESTION

Compare the feature branch with main and identify differences.

ANSWER / PROCEDURE

Compare the two branch tips and restrict the diff to the requested file. If the examiner asks for a true merge-base comparison, use the three-dot form.

Commands / configuration

```
git diff main feature/vehicle-update -- "vehicle.jsp"
git diff main...feature/vehicle-update -- "vehicle.jsp"
```

GUI route (Eclipse EGit): Right-click project → Team → use Switch/Checkout, Commit, Pull/Fetch, Merge, Rebase, Compare With, or Push. For advanced recovery use Git Bash.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q4(d) — TASK / QUESTION

Merge the vehicle-page feature into main, push, and verify the remote JSP.

ANSWER / PROCEDURE

Switch to main, make sure it is current, merge the feature branch, resolve conflicts if Git stops, then commit the merge if required.

Commands / configuration

```
git switch main
git pull origin main
git merge feature/vehicle-update
git status
git push origin main
```

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q5(a) — TASK / QUESTION

Create a Dockerfile for the Vehicle Rental Maven Web Application and choose a Java/Tomcat base image.

ANSWER / PROCEDURE

For a JSP/Servlet WAR, use Tomcat as the runtime. The WAR is copied into Tomcat's `webapps` directory. A multi-stage build is cleaner because Maven is used only in the build stage.

Commands / configuration

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9-jdk17
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY --from=build /app/target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

Q5(b) — TASK / QUESTION

Build the Maven WAR and copy it into Tomcat webapps in the image.

ANSWER / PROCEDURE

For a JSP/Servlet WAR, use Tomcat as the runtime. The WAR is copied into Tomcat's `webapps` directory. A multi-stage build is cleaner because Maven is used only in the build stage.

Commands / configuration

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9-jdk17
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY --from=build /app/target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show the terminal command and the successful phase/output. If you changed pom.xml, reopen it in Eclipse and show the exact changed element before rebuilding.

Q5(c) — TASK / QUESTION

Expose the Tomcat port and start Tomcat.

ANSWER / PROCEDURE

For a JSP/Servlet WAR, use Tomcat as the runtime. The WAR is copied into Tomcat's `webapps` directory. A multi-stage build is cleaner because Maven is used only in the build stage.

Commands / configuration

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM tomcat:9-jdk17
RUN rm -rf /usr/local/tomcat/webapps/ROOT
COPY --from=build /app/target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

GUI route: Eclipse → Window → Show View → Servers → add/start Tomcat; then open the URL in a browser and check the Servers/Console view for startup errors.

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

Q5(d) — TASK / QUESTION

Build vehiclerentalapp with a tag and verify it.

ANSWER / PROCEDURE

Run the build from the directory containing the Dockerfile and required build artifacts. Then verify the image ID, tag, and size.

Commands / configuration

```
docker build -t vehiclerentalapp:v1 .
docker images vehiclerentalapp
```

Examiner proof: perform the step, show the resulting file/GUI state, and verify the final outcome rather than stopping after the command.

Q6(a) — TASK / QUESTION

Run vehiclerentalapp with host 8080 mapped to the appropriate container port.

ANSWER / PROCEDURE

Start detached, map host 8080 to container 8080, and name the container so it is easy to demonstrate.

Commands / configuration

```
docker run -d --name vehicle-rental -p 8080:8080 vehiclerentalapp:v1
docker ps
docker port vehicle-rental
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

Q6(b) — TASK / QUESTION

Verify the running container and access localhost:8080.

ANSWER / PROCEDURE

A successful Maven build only proves the build lifecycle completed. Verify the WAR/JAR exists, start the server, confirm the server log says it started, confirm the expected port is listening, then open the correct context path. Check browser/server logs for 404/500 errors.

Commands / configuration

```
dir target
netstat -ano | findstr :8080
tasklist | findstr /I java
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

Q6(c) — TASK / QUESTION

Tag the image with your Docker Hub username, log in, and push it.

ANSWER / PROCEDURE

Authenticate, tag the local image with your Docker Hub namespace, push it, then verify on Docker Hub or with `docker pull` from another machine.

Commands / configuration

```
docker login
docker tag vehiclerentalapp:v1 <dockerhub-username>/VehicleRentalApp:v1
docker push <dockerhub-username>/VehicleRentalApp:v1
docker pull <dockerhub-username>/VehicleRentalApp:v1
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: run `git status` and/or `git log -1 --oneline`; show the relevant branch/remote/diff output. Do not only say the command — show its result.

Q6(d) — TASK / QUESTION

Verify the VehicleRentalApp image in Docker Hub.

ANSWER / PROCEDURE

Authenticate, tag the local image with your Docker Hub namespace, push it, then verify on Docker Hub or with `docker pull` from another machine.

Commands / configuration

```
docker login
docker tag vehiclerentalapp:v1 <dockerhub-username>/VehicleRentalApp:v1
docker push <dockerhub-username>/VehicleRentalApp:v1
docker pull <dockerhub-username>/VehicleRentalApp:v1
```

GUI route: Docker Desktop → Images/Containers → select the image or container → Run/Start/Stop/Logs/Open in terminal. Older versions use similar Images and Containers screens.

Examiner proof: show `docker images` or `docker ps -a`, then show logs/ports when relevant. For browser tasks, actually open `http://localhost:8080`.

6. EXAMINER DEMONSTRATION CHECKLIST

- Open the project folder and prove the repository/remote is correct.
- Open pom.xml and point to groupId, artifactId, version, packaging, dependencies and plugins when asked.
- Run mvn -version and, if required, java -version/javac -version.
- Build with mvn clean package (or the exact lifecycle command in the question).
- Show target/ and the generated WAR/JAR.
- In Eclipse, use Maven → Update Project when dependencies are stale.
- For Git, show status/diff before commit and log after commit.
- For rebase/merge questions, show the branch name and final clean status.
- For Docker, show docker images, docker ps -a, docker logs and port mapping as appropriate.
- Open http://localhost:8080 only after proving the server/container is running.
- For Docker Hub, show tag → login → push → repository verification.

7. OLD-VERSION FALLBACKS

If you see...	Use this fallback
Old Git without `git switch`	Use `git checkout -b branch` / `git checkout branch`.
Old Eclipse	Look under File → Import → Maven and Window → Show View → Servers; labels vary slightly.
Old Maven project	Check for `mvnw.cmd`; otherwise use a Maven version compatible with the project/JDK.
Old Tomcat project using javax.servlet	Prefer a javax-compatible Tomcat release (commonly Tomcat 9) unless the project is migrated to Jakarta.
Docker Desktop GUI differs	Use the CLI; the core objects remain images, containers, logs, and port mappings.
PowerShell command differs	Git Bash is the simplest universal shell for the Git commands in these papers.

Official references used for the practical workflow

Git for Windows: <https://git-scm.com/install/windows>

Eclipse documentation: <https://www.eclipse.org/documentation/>

Maven Compiler Plugin: <https://maven.apache.org/plugins/maven-compiler-plugin/>

Maven Dependency Plugin: <https://maven.apache.org/plugins/maven-dependency-plugin/>

Docker Desktop for Windows: <https://docs.docker.com/desktop/setup/install/windows-install/>